38.1 健康检查

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250k tokens，输出 ~5k tokens（本节）

健康检查是生产环境中不可或缺的基础设施——它让运维人员和自动化系统能够判断 Gateway 是否正常运行。OpenClaw 提供了多层次的健康检查机制：**Gateway 探针**（WebSocket 级别）、**健康 RPC 方法**（应用级别）和**状态快照**（系统级别）。

38.1.1 Gateway 探针（Probe）

`probeGateway()`（`src/gateway/probe.ts`，126 行）是 OpenClaw 最全面的健康检查工具。它不是简单的 TCP 端口检测，而是通过 WebSocket 建立一个完整的客户端连接，然后请求多项数据：

```
// src/gateway/probe.ts（简化）
export async function probeGateway(opts: {
  url: string;
  auth?: { token?: string; password?: string };
  timeoutMs: number;
}): Promise<GatewayProbeResult> {
  const client = new GatewayClient({
    url: opts.url,
    mode: "probe",            // 特殊的探针模式
    onHelloOk: async () => {
      // 握手成功后，并行请求四项数据
      const [health, status, presence, configSnapshot] = await Promise.all([
        client.request("health"),
        client.request("status"),
        client.request("system-presence"),
        client.request("config.get", {}),
      ]);
      // 返回结构化结果
    },
  });
  client.start();
}
```

返回的 `GatewayProbeResult` 包含：

字段	类型	含义
`ok`	`boolean`	整体是否健康
`url`	`string`	探测的 Gateway URL
`connectLatencyMs`	`number`	WebSocket 连接延迟（毫秒）
`error`	`string | null`	错误信息
`close`	`{code, reason}`	WebSocket 关闭码和原因
`health`	`HealthSummary`	频道和 Agent 的健康摘要
`status`	`object`	Gateway 运行状态
`presence`	`SystemPresence[]`	在线设备列表
`configSnapshot`	`object`	当前配置快照

探针模式（`mode: "probe"`）是 Gateway 客户端的特殊连接模式——它只进行只读查询，不会被计入活跃客户端计数，也不会触发聊天或命令执行。

38.1.2 健康 RPC 方法

Gateway 暴露了 `health` 和 `status` 两个 RPC 方法，可以通过 WebSocket 调用：

```ts
// src/gateway/server-methods/health.ts
export const healthHandlers: GatewayRequestHandlers = {
  health: async ({ respond, context, params }) => {
    const wantsProbe = params?.probe === true;
    const cached = context.getHealthCache();

    // 如果缓存有效且不要求主动探测，返回缓存
    if (!wantsProbe && cached && now - cached.ts < HEALTH_REFRESH_INTERVAL_MS) {
      respond(true, cached, undefined, { cached: true });
      // 后台异步刷新缓存
      void context.refreshHealthSnapshot({ probe: false });
      return;
    }

    // 否则同步刷新并返回
    const snap = await context.refreshHealthSnapshot({ probe: wantsProbe });
    respond(true, snap, undefined);
  },

  status: async ({ respond }) => {
    const status = await getStatusSummary();
    respond(true, status, undefined);
  },
};
```

健康缓存策略

健康检查涉及对多个频道（如 Telegram、Discord）的探测，可能耗时数秒。为了避免频繁查询影响性能，Gateway 实现了两层缓存：

1. **内存缓存**： `healthCache` 变量保存最新的 `HealthSummary` ，带有时间戳和版本号
2. **后台刷新**：当返回缓存结果时，同时在后台触发异步刷新——用户看到的是即时响应，同时缓存在后台静默更新

`refreshGatewayHealthSnapshot()` 使用 Promise 去重——如果已有刷新正在进行，新的刷新请求会等待同一个 Promise 完成，避免并发重复探测：

```
// src/gateway/server/health-state.ts
export async function refreshGatewayHealthSnapshot(opts?: { probe?: boolean }) {
  if (!healthRefresh) {
    healthRefresh = (async () => {
      const snap = await getHealthSnapshot({ probe: opts?.probe });
      healthCache = snap;
      healthVersion += 1;
      broadcastHealthUpdate?.(snap);    // 通知所有连接的客户端
      return snap;
    })().finally(() => { healthRefresh = null; });
  }
  return healthRefresh;  // 多个请求共享同一个 Promise
}
```

38.1.3 健康摘要结构

`getHealthSnapshot()` （`src/commands/health.ts`，788 行）生成全面的 `HealthSummary`，包含：

频道健康

对每个已配置的频道（Telegram、Discord、WhatsApp 等），检查：

- **是否已配置**：API 密钥/凭证是否存在
- **是否已链接**：OAuth 令牌是否有效
- **探测结果**：主动调用频道 API 验证连通性
- **Bot 信息**：bot 用户名、webhook URL

Agent 健康

对每个配置的 Agent：

- **心跳状态**：`HeartbeatSummary`（间隔时间、上次运行时间）
- **会话统计**：会话数量、最近活跃的会话列表

会话健康

- **会话存储路径**
- **会话总数**
- **最近 5 个活跃会话**（键名、更新时间、活跃时长）

38.1.4 状态快照

`buildGatewaySnapshot()` （`src/gateway/server/health-state.ts`）在 Gateway 启动时和客户端首次连接时生成初始状态快照：

```
export function buildGatewaySnapshot(): Snapshot {
  const cfg = loadConfig();
  return {
    presence: listSystemPresence(),          // 在线设备
    health: {},                               // 占位，异步填充
    stateVersion: {
      presence: presenceVersion,              // Presence 变更计数器
      health: healthVersion,                  // 健康数据变更计数器
    },
    uptimeMs: Math.round(process.uptime() * 1000),  // Gateway 运行时间
    configPath: CONFIG_PATH,                  // 实际配置文件路径
    stateDir: STATE_DIR,                      // 状态目录路径
    sessionDefaults: {
      defaultAgentId,                         // 默认 Agent ID
      mainKey,                                // 主会话键
      mainSessionKey,                         // 主会话标识
      scope,                                  // 会话作用域模式
    },
  };
}
```

状态版本号（`presenceVersion` 和 `healthVersion`）是单调递增的计数器，客户端可以通过比较版本号判断是否需要刷新本地缓存——这比轮询完整数据更高效。

38.1.5 心跳广播

Gateway 定期向所有已连接的客户端广播心跳事件，包含最新的健康和 Presence 数据。当健康状态发生变化时，`broadcastHealthUpdate()` 也会触发实时推送：

```
Gateway ──heartbeat──→ [所有客户端]
         {health, presence, stateVersion}

Gateway ──health.update──→ [所有客户端]   (健康变更时)
         {HealthSummary}
```

这种推送模式确保了 Web 控制台和原生应用能实时反映 Gateway 的健康状态，无需客户端主动轮询。

本节小结

1. **Gateway 探针**是最全面的健康检查——它建立完整 WebSocket 连接，并行请求健康、状态、Presence 和配置四项数据，适用于外部监控和 CLI 诊断。
2. **健康 RPC 方法**支持缓存 + 后台刷新的双层策略——缓存有效时立即返回，同时异步更新；Promise 去重避免并发探测。
3. **健康摘要**覆盖频道（探测 + 配置）、Agent（心跳 + 会话）和系统（运行时间 + 版本）三个维度。
4. **状态版本号**是高效的缓存失效机制——单调递增计数器让客户端只需比较数字即可判断是否需要刷新。
5. **心跳广播**将健康数据主动推送到所有客户端，实现实时状态同步。